



UNIVERSIDADE FEDERAL DO CEARÁ

CENTRO DE TECNOLOGIA

DEPARTAMENTO DE ENGENHARIA DA COMPUTAÇÃO

PROCESSAMENTO DE IMAGENS E SISTEMAS EM NUVEM

SEMESTRE 2026.1

Sistema de Reconhecimento Facial Distribuído

Augusto Ferreira de Freitas - 472144

Arthur Bernardo Oliveira da Costa - 563535

Gabriel Miranda dos Santos - 497314

João Lucas Oliveira Mota - 509597

Sumário

1	Introdução	3
2	Arquitetura Distribuída e Protocolos de Comunicação	4
2.1	Topologia da Rede e Desacoplamento de Nós	5
2.2	Base de Dados de Validação e Protocolo de Experimentos	5
2.3	Mecanismo de Consulta por similaridade vetorial	6
2.4	Implementação com a leitura de Rostos em Tempo Real	6
3	Nó Cliente	7
3.1	Pipeline de Visão Computacional:	7
3.2	Isolamento da Região de Interesse (ROI):	7
3.3	Reconhecimento Facial com DeepFace e ArcFace:	7
4	Resultados e Discussão	9
4.1	Avaliação Técnica:	9
4.2	Conclusão:	9

1 Introdução

Contextualização: O desafio de automatizar o registro de ponto via reconhecimento facial, eliminando a necessidade de crachás físicos.

Abordagem Proposta: Uso de processamento local na borda (Edge) para filtragem e serviços em nuvem (Serverless) para processamento inteligente.

2 Arquitetura Distribuída e Protocolos de Comunicação

O sistema proposto foi modelado sob o paradigma cliente-servidor fracamente acoplado, dividindo a carga de trabalho entre o processamento de borda (*edge computing*) e o armazenamento especializado em nuvem. Essa separação de responsabilidades buscou otimizar o uso dos recursos computacionais locais e garantir a escalabilidade horizontal do banco de dados de vetores. A disposição estrutural de cada componente e a interconexão dos blocos estão apresentadas na Figura 1.

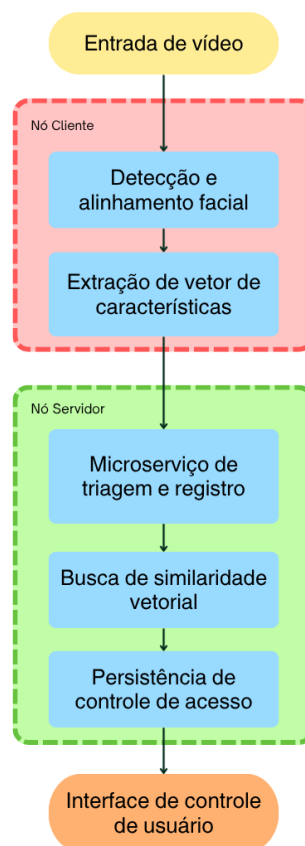


Figura 1: Fluxo de dados e divisão de componentes entre as camadas de Cliente e Servidor em nuvem.

Conforme observado no diagrama, o fluxo operacional iniciou no ambiente local com a captura de vídeo pelo dispositivo de entrada e o processamento da imagem via OpenCV para a detecção e o alinhamento facial. Na sequência, o modelo ArcFace extraiu os vetores descritores de 512 dimensões. Ao atingir a camada em nuvem, o microserviço AWS Lambda realizou a

validação e o registro temporal (*timestamp*) da requisição, acionando o cluster do Qdrant Cloud para a execução da busca vetorial. Após a localização do vetor mais próximo, o histórico de acesso foi persistido no banco de dados relacional Amazon RDS, o que atualizou o status de liberação na interface de controle de acesso.

2.1 Topologia da Rede e Desacoplamento de Nós

A topologia do sistema é composta por dois nós principais que operam de forma geograficamente distribuída:

- **Nó Cliente (Local):** Executa em ambiente computacional próprio, encarregado das tarefas de computação intensiva na origem, que compreendem a leitura do sistema de arquivos local, a decodificação de matrizes de imagens e a geração dos vetores de características de 512 dimensões.
- **Nó Servidor (Nuvem):** Consiste em um cluster gerenciado do *Qdrant Cloud*, hospedado na infraestrutura da *Amazon Web Services* (AWS) na região da América do Sul (*sa-east-1*). Este nó é responsável exclusivo pela indexação espacial, persistência e processamento de consultas de similaridade de alta dimensionalidade.

A comunicação entre as extremidades ocorre por meio de requisições HTTP utilizando uma interface de programação de aplicações baseada na arquitetura REST (*RESTful API*), implementada sobre as bibliotecas de transporte `httpx` e `httpcore`.

2.2 Base de Dados de Validação e Protocolo de Experimentos

A validação experimental da infraestrutura distribuída foi conduzida utilizando a base de dados de faces da FEI (*FEI Face Database*) [Thomaz and Giraldi 2006]. Este repositório biométrico é composto por 2.800 imagens coloridas com resolução original de 640×480 pixels, obtidas a partir de um grupo de 200 indivíduos, apresentando uma divisão equitativa de 100 sujeitos do sexo masculino e 100 do sexo feminino, com idades variando entre 19 e 40 anos. Cada participante possui 14 registros fotográficos capturados contra um fundo branco homogêneo, abrangendo diferentes expressões faciais e rotações de perfil de até cerca de 180 graus, com variações de escala estimadas em 10%.

No modelo de testes aplicado ao sistema distribuído, o conjunto de dados foi segmentado e alocado logicamente para simular um cenário real de identificação remota:

- **Fase de Ingestão:** O subconjunto composto pelas imagens em posição estritamente frontal e com expressão neutra permaneceu no nó cliente para o processo de extração local e alimentação síncrona inicial da coleção hospedada no *Qdrant Cloud*.
- **Fase de Consulta:** As imagens que apresentavam variações de pose, rotação e expressões do mesmo indivíduo foram isoladas em um diretório de testes secundário no nó local. Esses arquivos foram utilizados para alimentar o laço contínuo de requisições externas, validando a capacidade do método remoto `query_points` em correlacionar vetores com distorções geométricas ao vetor original armazenado na nuvem.

2.3 Mecanismo de Consulta por similaridade vetorial

A etapa de consulta biométrica e identificação no banco de dados vetorial foi realizada por meio de uma chamada de procedimento remoto utilizando o método unificado `query_points` da API do *Qdrant*. Este método consolida a pesquisa espacial em uma única requisição estruturada, otimizando o tempo de resposta do cluster em nuvem.

O vetor de características composto por 512 dimensões, gerado na borda pelo modelo *Arc-Face*, é encaminhado como o argumento principal da consulta no parâmetro `query`. Configura-se o parâmetro `limit` com valor unitário para estabelecer uma busca do tipo *top – 1*, o que instrui o servidor a retornar apenas o vetor que apresenta a maior proximidade espacial. A requisição também define o parâmetro `with_payload` como verdadeiro, garantindo que o servidor anexe os metadados associados ao registro localizado.

No servidor, o *Qdrant* processa a busca calculando a similaridade de cosseno entre o vetor de entrada e os elementos indexados na coleção. O retorno gerado pelo método é estruturado em uma lista sob o atributo `points`. O sistema analisa o primeiro elemento dessa lista para extrair o identificador único do ponto, os metadados da imagem original e o escore de similaridade correspondente, o qual serve como métrica de confiança para a validação da identidade.

2.4 Implementação com a leitura de Rostos em Tempo Real

3 Nú Cliente: Pipeline de Processamento de Imagens e Extração de Características

3.1 Pipeline de Visão Computacional:

Descrição das Etapas locais de tratamento da imagem

3.2 Isolamento da Região de Interesse (ROI):

Uso de técnicas de detecção facial com a yolo para extrair a região do rosto, garantindo que o processamento subsequente seja focado apenas na área relevante.

3.3 Reconhecimento Facial com DeepFace e ArcFace:

O DeepFace é utilizado como uma biblioteca de apoio para facilitar a integração de modelos de reconhecimento facial baseados em aprendizado profundo. Ele evita a necessidade de implementar manualmente toda a estrutura de reconhecimento facial, oferecendo recursos prontos para etapas como detecção, alinhamento, representação e verificação de faces. [Du et al. 2022]. Além disso, por funcionar como uma estrutura híbrida, permite utilizar diferentes modelos já conhecidos na área, como VGG-Face, FaceNet, OpenFace, DeepID, Dlib e, especificamente nesse caso, o ArcFace. [Serengil and contributors 2026].

Nesse projeto, o ArcFace é utilizado por meio do DeepFace para converter cada face capturada em um vetor numérico, chamado de embedding facial. Esse vetor representa características relevantes do rosto e permite que a comparação entre pessoas seja feita de forma matemática, em vez de comparar diretamente as imagens. A principal contribuição do ArcFace está na função de perda chamada *Additive Angular Margin Loss*, que melhora a separação entre diferentes identidades no espaço vetorial. Enquanto modelos tradicionais baseados em softmax aprendem principalmente a classificar imagens em categorias, o ArcFace busca gerar representações mais discriminativas, aproximando embeddings de uma mesma pessoa e afastando embeddings de pessoas diferentes por meio de uma margem angular aditiva. Dessa forma, a comparação facial passa a considerar a separação entre os vetores em um espaço angular, tornando o reconhecimento mais adequado para cenários com variações de iluminação, pose e

expressão facial. [Deng et al. 2019]

A formulação do ArcFace parte da normalização dos pesos da última camada e dos vetores de características, de modo que a comparação passe a depender principalmente do ângulo entre os vetores. A função de perda pode ser representada, de forma resumida, por:

$$L = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j=1, j \neq y_i}^n e^{s \cos(\theta_j)}}$$

Nessa expressão, N representa o número de amostras, y_i é a classe correta da amostra i , θ_{y_i} é o ângulo entre o vetor de características e o peso da classe correta, m é a margem angular adicionada e s é um fator de escala. A adição da margem m força o modelo a exigir uma separação angular maior para classificar corretamente uma face, tornando os embeddings mais discriminativos [Deng et al. 2019].

No funcionamento do sistema, após o pré-processamento realizado pelo script de captura da ROI descrito anteriormente, a região facial é processada pelo DeepFace utilizando o ArcFace como modelo de representação. O resultado dessa etapa é um embedding facial, isto é, um vetor numérico que resume as características relevantes da face capturada. Em vez de comparar imagens diretamente, o sistema compara esse vetor com os vetores previamente cadastrados no banco.

4 Resultados e Discussão

4.1 Avaliação Técnica:

Eficiência do algoritmo geométrico local frente a variações de iluminação e latência média do processo na nuvem.

4.2 Conclusão:

Benefícios do modelo híbrido em termos de escalabilidade, custo sob demanda e economia de recursos de infraestrutura.

Tabela 1: Template de tabela

Métrica	Classe Normal	Classe Anomalia
Precision	0.99	0.99
Recall	0.99	0.99
F1-Score	0.99	0.99
Support	4034	3524

Referências

- [Deng et al. 2019] Deng, J., Guo, J., Xue, N., and Zafeiriou, S. (2019). Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4690–4699.
- [Du et al. 2022] Du, H., Shi, H., Zeng, D., Zhang, X.-P., and Mei, T. (2022). The elements of end-to-end deep face recognition: A survey of recent advances. *ACM Computing Surveys*, 54(10s):1–42.
- [Serengil and contributors 2026] Serengil, S. I. and contributors (2026). Deepface: A lightweight face recognition and facial attribute analysis framework for python. <https://github.com/serengil/deepface>. Acesso em: 31 maio 2026.
- [Thomaz and Giraldi 2006] Thomaz, C. E. and Giraldi, G. A. (2006). FEI face database. Acesso em: 2 de junho de 2026.

Apêndice A — Descrição Completa do Dataset